Placement Portal Application

Project Report

MAD-1 Jan 2026

1. Student Details

Student Name	Navon John Lukose
Student ID	24f2003934
Student Email	24f2003934@ds.study.iitm.ac.in
Project Title	Placement Portal Application
Course / Term	MAD-1 Jan 2026
Repository Note	24f2003934/mad1-placement

2. Project Details and Approach

Problem statement. Institutes need a system to manage company onboarding, placement drives, student applications, shortlisting, and historical placement records without relying on spreadsheets or email chains. This project implements a role-based Placement Portal Application with Admin, Company, and Student workflows.

Approach. The application was built as a server-rendered Flask monolith to keep the implementation reliable and viva-friendly. The design separates:

- authentication and role handling,
- role-specific blueprints for admin, company, and student flows,
- SQLAlchemy models for persistence,
- service functions for approval logic, application flow, and authorization checks.

Implemented core flows.

- Pre-seeded admin login
- Company and student registration
- Admin approval / rejection for companies and drives
- Company drive creation, editing, closing, deletion, and application status updates
- Student drive browsing, application submission, duplicate-application prevention, status tracking, and placement history
- Historical placement tracking for admin review
- Initial JSON API endpoints for the same domain entities

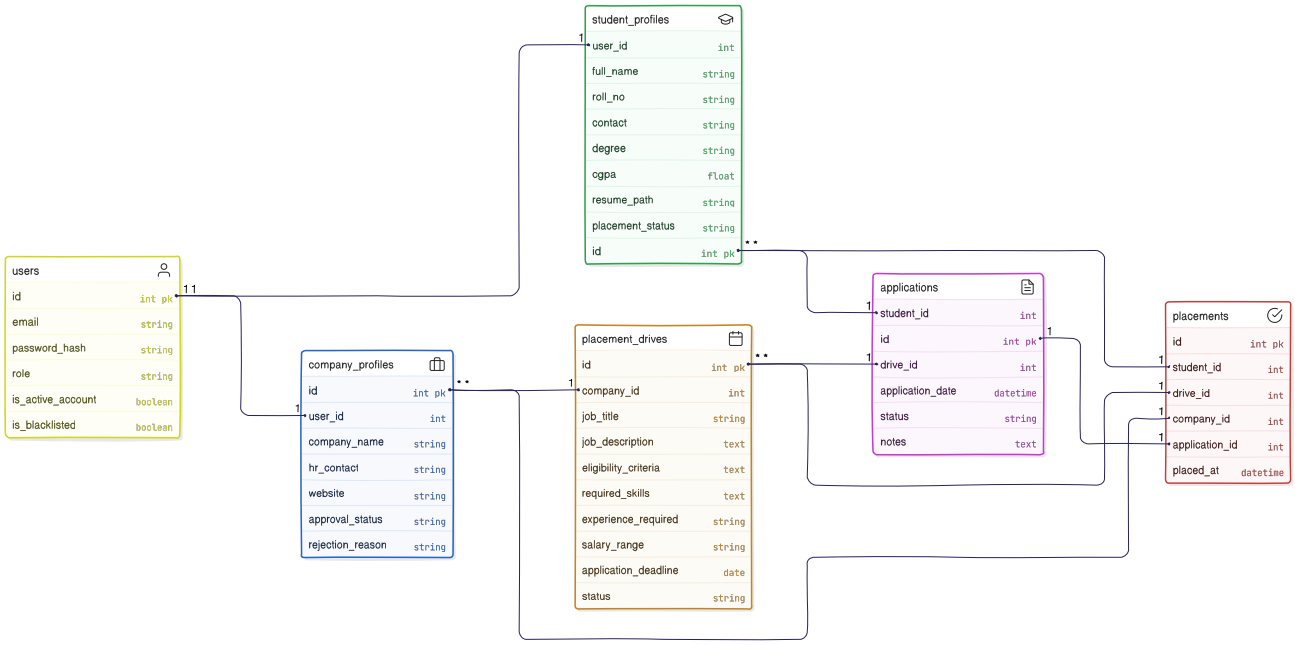
3. AI / LLM Declaration

Next to none AI / LLM assistance was used in this project. ChatGPT was used only as a reference aid for library/API usage. The implementation, testing, debugging, and final submission preparation were carried out primarily by the student.

4. Frameworks and Libraries Used

- Flask: application backend and routing
- Jinja2: server-side templating
- HTML/CSS/Bootstrap: frontend rendering and layout
- SQLite: relational database
- Flask-Login: session-based authentication
- Flask-SQLAlchemy: ORM and DB access
- Flask-WTF / WTForms: form handling and validation
- Werkzeug security helpers: password hashing and verification
- Pytest: automated testing

5. ER Diagram



6. API Resource Endpoints

Implemented JSON API endpoints under '/api':

Method	Endpoint	Purpose
POST	/api/auth/login	Session-based API login
POST	/api/auth/logout	Session logout
POST	/api/auth/register/student	Student registration
POST	/api/auth/register/company	Company registration
GET	/api/users	Admin list of users
GET / PATCH	/api/users/{id}	Admin user detail/update
GET	/api/companies	Admin list of companies
PATCH	/api/companies/{id}	Admin company approval update
PATCH	/api/companies/{id}/approval	Admin company approval update
GET	/api/drives	Role-aware drive listing
GET / PATCH / DELETE	/api/drives/{id}	Drive detail/update/delete
PATCH	/api/drives/{id}/approval	Admin drive approval update
POST	/api/drives/{id}/apply	Student applies to drive
GET	/api/applications	Role-aware application listing
GET / PATCH	/api/applications/{id}	Application detail/status update
GET / PATCH	/api/company/profile	Company profile read/update
GET / PATCH	/api/student/profile	Student profile read/update
GET	/api/placements	Role-aware historical placement listing

7. Presentation Video Link

Public video link: https://magniquick.github.io/iit-link